Clase 4 · strerror, perror, fork, huérfanos y la familia exec

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 3 sep 2026 · Tema 1 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. strerror() y perror()

- **strerror(n):** devuelve la cadena del mensaje de sistema para el código n (`str` = string).

```
for (i = 0; i <= 150; i++) printf("%d: %s\n", i, strerror(i));
```

- En Fedora el último código válido es el **133**; desde el 134, *Unknown error*. El **41** está reservado. El **0** = éxito (*Success*), no error.
- **perror("txt"):** si la instrucción anterior falló, imprime `txt: <mensaje del sistema>` (P = personalizar). Ej.: `perror("fork")`.

2. Control de flujo de fork()

Tres caminos: **error (−1)**, **hijo (0)**, **padre (PID del hijo)**.

```
if ((child = fork()) == -1) { ... }   /* 1) fork  2) asigna  3) compara */
pid_t ch; ch = fork(); if (ch == -1) { ... }   /* forma larga, más clara */
```

- Documentación oficial: Open Group. `fork`, `getpid`, `getppid`, `getuid`... en `<unistd.h>`; `pid_t` es entero (`%d`).
- Causa más común de −1: **falta de memoria** para duplicar la imagen; se fija `errno`.
- `exit(EXIT_FAILURE)` ≡ exit(1); `EXIT_SUCCESS` ≡ exit(0).

Error común: confundir `=` con `==` dentro del paréntesis.

3. Procesos huérfanos y reapers

- programa08: sale "in parent" antes que "in child" y el hijo obtiene `getppid()` = **1**.

> **Huérfano (examen)**
>
> "Un proceso cuyo padre ya murió, pero él sigue vivo." Lo adopta **PID 1** (init o systemd). En distros modernas hay **reapers** que adoptan huérfanos para descargar al PID 1, por eso a veces el PPID no es 1.
>
> - `cat /etc/os-release` → Fedora Linux Workstation 40. Jerarquía: hijo, padre (llamó a fork), abuelo (la terminal).

4. Familia exec

Seis funciones; las letras indican el comportamiento:

Letra	Significado
l (list)	Argumentos en lista separada por comas, terminada en NULL o `(char *)0`
v (vector)	Argumentos en arreglo de punteros terminado en NULL
e (environment)	Vector propio de variables de entorno
p (PATH)	Busca el ejecutable en PATH: basta el nombre

- `execl`, `execv`, `execle`, `execve` → **path** (ruta absoluta `/bin/ls`). `execlp`, `execvp` → **file** (`ls`).
- Si tiene éxito, **reemplaza la imagen** del proceso (no crea PID nuevo); el original no recupera el control.
- **Argumento cero:** `argv[0]` repite el nombre: `execl("/bin/ls", "ls", "-l", "/", (char *)0);`

Manejo de errores obligatorio

```
if (execl("/bin/ls", "ls", "-l", "/", (char *)0) == -1) {
    perror("execl");
    exit(EXIT_FAILURE);
}
```

Si exec funciona, el código de abajo es **inalcanzable**.

5. Demo programa10.c

- Con `/bin/ls`: lista `/` y no aparece "esto no se imprime".
- Con `/bin/lsa` y el `exit` comentado: `execl: No such file or directory` y **sí** imprime "esto no se imprime": el proceso sobrevivió por no manejar el error.

Cierre

Sin tarea formal; tarea moral: repasar la familia exec (sobre todo `execl`). Martes: Tema 2, estados de proceso. CentOS descontinuado → AlmaLinux o Rocky Linux.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
strerror / perror	Mensaje del código / `txt: mensaje`
fork	−1 error, 0 hijo, PID al padre
Huérfano	Padre muerto; lo adopta PID 1 o un reaper
exec l / v / e / p	Lista / vector / entorno / busca en PATH
exec exitoso	Reemplaza la imagen; lo de abajo no se ejecuta
argv[0]	Repite el nombre del ejecutable
Si exec falla	`perror` + `exit(EXIT_FAILURE)`